

PROJECT 4: GRPC and Distributed Systems

1 FOREWORD

Project 4 is designed to make students familiarize with remote procedure calls and the distributed file system (DFS). This project contains two parts. Part 1 is to introduce the gRPC c++ API and Protocol Buffers. You will employ them to build a simple file sharing system using synchronous remote procedure calls. Part 2 builds on part 1 and takes a step further to construct a remote distributed file system (DFS) with multiple clients. In this part, you will get a taste of asynchronous requests and a weakly consistent synchronization system to manage cache consistency between multiple clients and a single server.

2 PART ONE: BUILDING THE RPC PROTOCOL SERVICE

2.1 Project description

In this part, you will build a simple file sharing system. The gRPC c++ API is employed to implement the remote procedure calls (RPC), and Protocol Buffers is used as definition language.

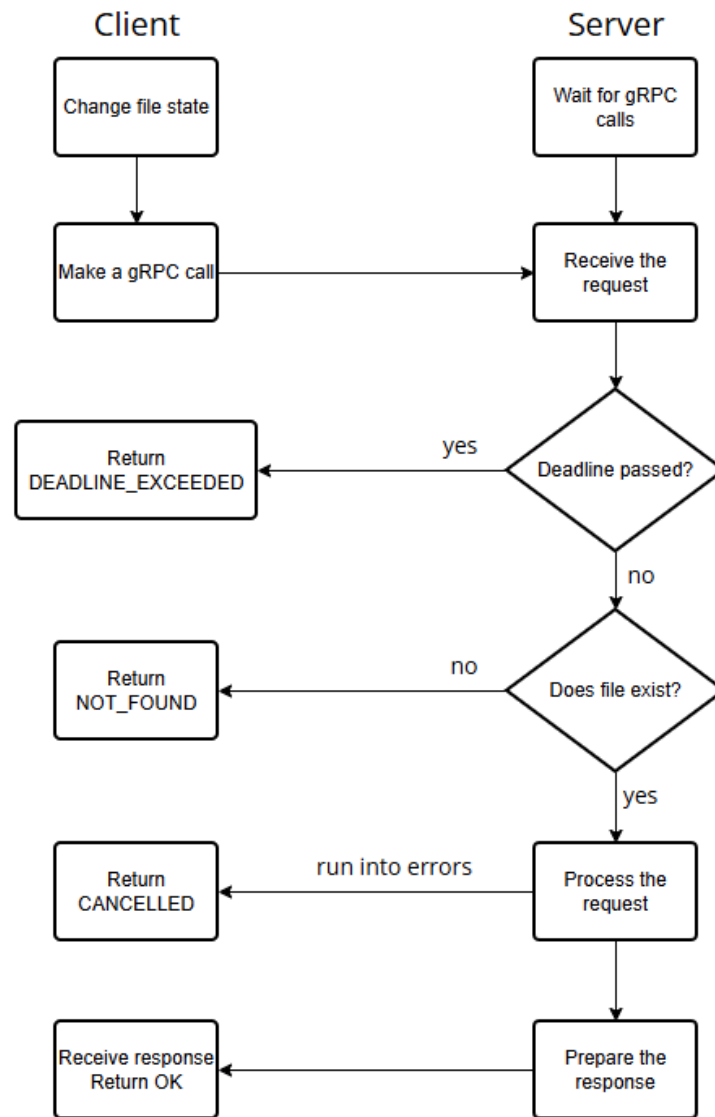
There are five RPC services to be implemented: The Store method stores a local file at the client side to a remote server; The Fetch method requests a file transfer from a remote server; The Delete method removes a specific file at a remote server; The List method requests a complete file list from a remote server; The Stat method retrieves file attributes of a requested file from a remote server, such as size, created time, and modified time.

2.2 Flow of control

In part 1, this file sharing system is designed for one client and one server pair to share files. Since it is one to one connection with only synchronous remote procedure calls, the relationship between client and server is straightforward.

Once a client is connected to a server, it makes any RPC service request. Then, the client is blocked and waits for the response from the server. Based on the return gRPC status the server replies, the client takes appropriate moves. Similarly, the server starts and waits for any gRPC call from the client, processes the requests and sends back appropriate responses, and then is blocked to wait for further requests. Based on the result of procedure calls, the server returns

gRPC status (either OK or error with error code). Client will process the error code and return an appropriate *StatusCode* as *OK*, *NOT_FOUND*, *CANCELLED*, or *DEADLINE_EXCEEDED*.



This system is simple and reliable, and doesn't need any additional synchronous calls as they both synchronize with each other as processing the gRPC calls. But the trade off is both server and client are blocked by the synchronous calls. This can lead to inefficiencies if the server or client spends significant time waiting for I/O or computation. The one-to-one, blocking model does not scale well to support multiple clients. A more advanced architecture (e.g., asynchronous or multi-threaded) would be required for concurrent connections, which we will discuss more at part 2.

2.3 Implementation

For the Store method, the client sends a stream of file content along with a file name and a file size. The client uses a *ClientWriter* to send the stream, while the server uses a *ServerReader* to receive it. It runs in a while loop to continually send content chunks and check the *bytes_sent* with *filesize* to verify the correctness of the file transfer.

The Fetch method takes in a request with a file name and returns a stream of file chunks. Similar to the Store, it runs in a while loop to ensure all of the file contents are fetched, and it checks the file size to confirm the file transferring completion, except that the client uses a *ClientReader* and the server uses a *ServerWriter* to complete the mission.

The Delete method removes a specific file at a remote server as the client sends a *filename* and the server returns a boolean value of *success*.

The List method requests a complete file list from a remote server as the client sends an *Empty* object and the server returns a list of *filename* and *mtime*. The server opens the file directory and reads in a while loop to include all files in the directory. Once the client receives the response, it creates a simple map of files and inserts the objects using *filename* as key and *mtime* as value.

The Stat method retrieves file attributes of a requested file from a remote server, such as *filename*, *size*, *ctime*, and *mtime*.

For all methods, gRPC status is returned based on the process result. The client needs to use the ok status to return *StatusCode::OK*, or error code to return *StatusCode::NOT_FOUND*, *CANCELLED*, or *DEADLINE_EXCEEDED* depending on the condition.

Additionally, the client sets up a *deadline* of each request in *ClientContext* and passes it to the server. The server checks if it passes the deadline by invoking *IsCancelled()* before processing the request.

2.4 Testing

Testing includes:

- Test each gRPC method with an existing filename and a reasonable deadline, expecting an *OK* code.
- Test Store, Fetch, Delete, and Stat method with a filename which doesn't exist, expecting a *NOT_FOUND* code.

-Test Store and Fetch method with a large file and a short deadline, expecting a *DEADLINE_EXCEEDED* code.

-Test Delete method with directory not existing, expecting a *CANCELLED* code.

3 PART TWO: COMPLETING THE DISTRIBUTED FILE SYSTEM

3.1 Project description

In this part, we will design a distributed file system, which can handle multiple clients with write/read access. Clients use cache mechanisms to store files locally. Any change of file states on the client side is required to be updated to the server, and vice versa. Therefore, we need to design a synchronization model to maintain file consistency through the server and all clients.

On the client side, a watcher thread and an async thread are running concurrently alongside the working thread. The watcher thread is monitoring the state of the file directory. It will notify the system when any file creation, modification, or deletion occurs, and then invoke *Store* or *Delete* gRPC call to notify the server to update its state.

Async thread will send a *CallbackList* request to the server and wait for the response. Whenever the server has any state change, it will send back a list of files at the most updated versions to any client who sent an async call before. Once a client receives the list, it will iterate through the file list and compare its version with a locally cached file and call *Store* or *Fetch* accordingly.

In order to prevent any race condition on client side, server side, and client-server communications, we have several design decisions to make. On the client side, to keep the file directory clean and consistent while writing and reading, we need to avoid the watcher thread and the async thread to modify the file directory at the same time. Thus, we need a directory mutex to allow only one of the threads to touch the directory at a certain period of time.

On the server side, things are more complicated when multiple clients try to read or write to the files concurrently. First, there should be only one client allowed to write to a certain file at once. A writer lock system is designed for file writing access, meaning the client needs to acquire the writer lock via a gRPC call before writing to the file. Since each file in the system needs a writer lock, the server must maintain a lock map to keep track of the writer lock ownership. The lock map needs to have the lock information, such as *filename* and who has it (*clientID*). Then, to keep the map access clean, we have a map lock to prevent race conditions of lock access.

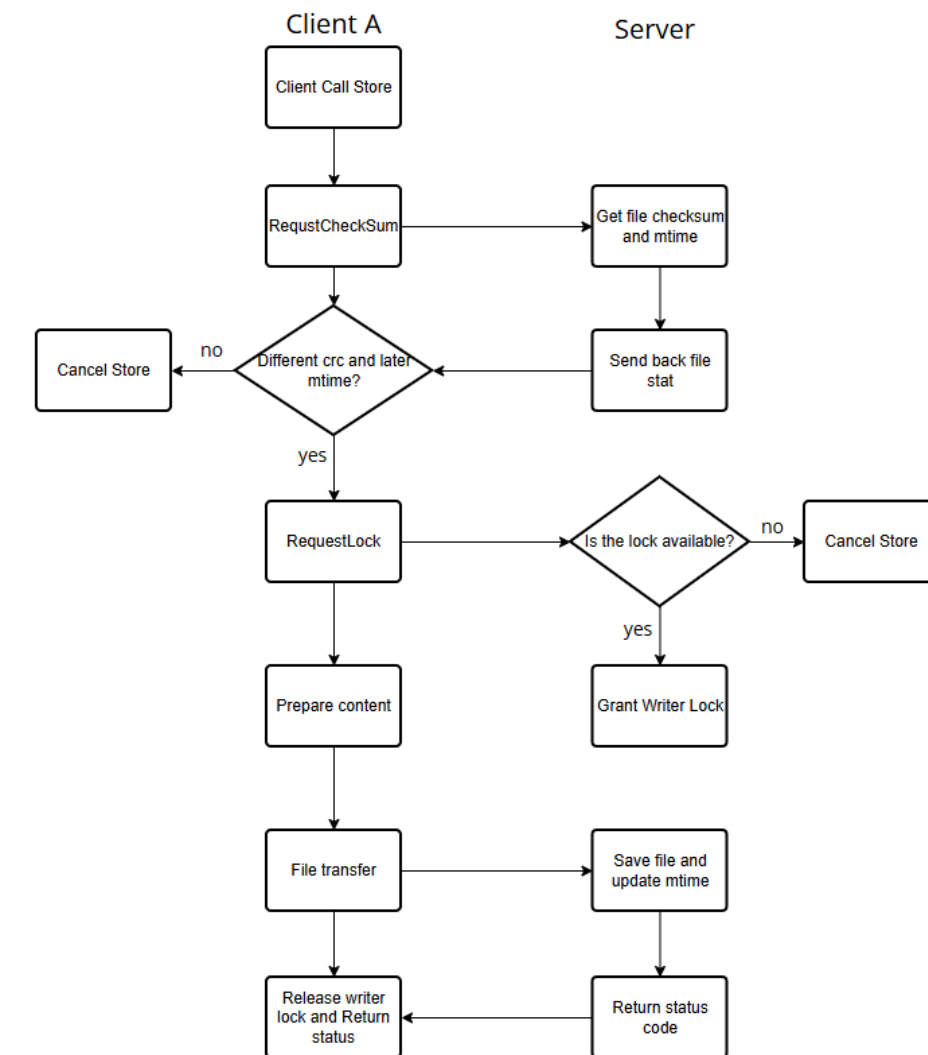
Besides the writer access, clients may access the file only for the reading, such as *Fetch*, *List*, and *Stat* gRPC calls. In those cases, the server should grant them reader access and allow multiple

clients to read the same file simultaneously to improve concurrency and reduce latency. Thus, each file should have a shared lock for different writer/reader scenarios, and the server should maintain a shared lock map for the file system and a map mutex for the map access.

Lastly, to properly handle the async calls from clients, the server must create a request queue to store all async requests and a mutex for the queue access. Thus, when the state of the file system changes, the server can send out the updated list based on the request queue.

3.2 Flow of control

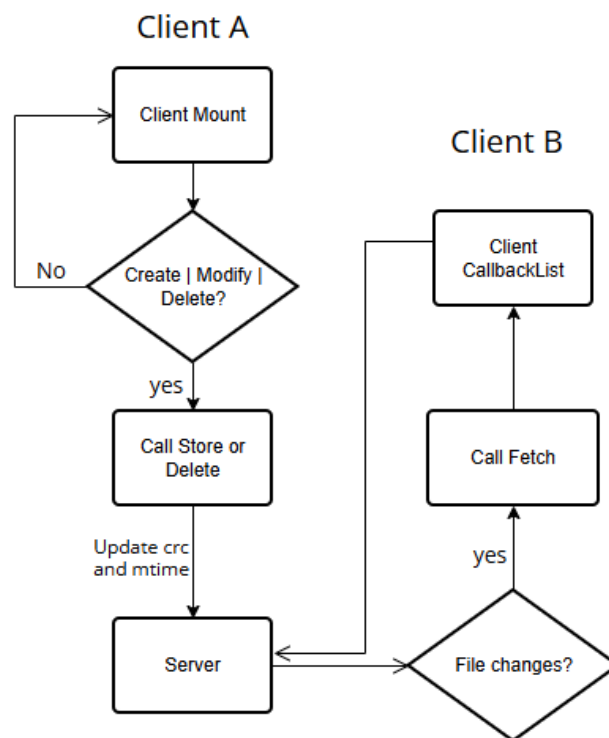
Here is the example flow chart for a scenario when a client calls the *Store* gRPC call to the server. Other gRPC calls are rather similar with file change or simpler without file change.



In addition to the gRPC calls implemented at part one, we must add synchronization procedures to keep the file system consistent. Thus, any move to change the server state will have to compare the file checksum and modified time with the server version to see if it is necessary to store the file into the remote file system. For this part, I decided to do the comparison at the client side to reduce server latency and implemented a gRPC call *RequestCheckSum* to obtain *crc* and *mtime* from the server to do the comparison locally. Only the file with different *crc* and larger *mtime* value (newer version) will trigger the next step which is to receive the writer access to the file from the server. The client will call *RequestLock* for this purpose.

Once the client receives the lock, it will transfer the file content and file attributes like *filesize* to verify the integrity of the files when the transfer is completed. It is fairly important to update the *mtime* of remote file so that both *crc* and *mtime* will be consistent through the server and all the clients later with async calls. The server will return the *grpc Status::OK* when it finishes the file receiving and writing to the remote directory. The client will verify the status code and call *ReleaseLock* to free the writer access before returning *StatusCode::OK*.

The following flow chart illustrates the synchronization process of watcher thread and async thread.



Both watcher thread and async thread at client side are initiated when client receives “mount” command. The watcher thread will call *InotifyWatcher* to monitor the *mout_path* directory and invoke *InotifyEventCallback* when it finds any *IN_CREATE* | *IN_MODIFY* | *IN_DELETE* event flag. The *InotifyEventCallback* will trigger Store or Delete gRPC call according to the event type. The following sync gRPC calls are described above.

The async thread will invoke *HandleCallbackList* and wait for the gRPC asynchronous callbacks from the server. The client node takes care of the first *InitCallbackList* request. When the server notices any state change, it will send out the updated file list via *ProcessQueuedRequests* to any client in the queue. The client will iterate through the list and compare *crc* and *mtime* with local files and trigger Fetch, Delete, or Store accordingly. Once it is completed, it initiates another *InitCallbackList* call for the next update. With this mechanism, the client is able to keep updated with the newest versions of local cached files.

3.3 Implementation

There are a few modifications for the sync gRPC calls. To fulfill the synchronization purpose, *mtime* is added into any file content transfer to keep an updated file with the same *mtime*. To prevent race conditions of file modifications through multiple clients, a writer lock is introduced for each file and a *RequestLock* and a *ReleaseLock* gRPC functions are designed for a client to apply the writer access. A *writer_locks* map is employed to store the information of each writer lock with its *filename* and *clientID* indicating the ownership, and a *writer_mutex* is used to control the map access. In addition, a *RequestChecksum* gRPC function is implemented for clients to compare *crc* and *mtime* before calling *Store* or *Fetch*. It makes sure all *Store* and *Fetch* calls are necessary. There is another way to design this procedure. For instance, when client calls *Store* gRPC call, client sends the *crc* and *mtime* along with the *Store* and let server decide whether to update the file or not. In this way, only one gRPC call is needed. But the server has to do an extra step of comparison and content transferring may be unnecessary if the local file is outdated.

The server is required to deal with multiple clients concurrently, meaning the system needs to control the writing/reading access for each file. Thus, *shared_timed_mutex* is used as *file_mutex*, which can perform *shared_lock* for reader access or *unique_lock* for writer access. A *file_mutex_map* is designed to keep track of each *file_mutex*, and a *map_mutex* used to keep the map access clean. When the server starts, the *FileMutexMapInit* function is invoked to initiate the *file_mutex_map*. The server uses *GetFileMutex* function to obtain the mutex if it exists or initiates a new one when the file is created, and uses *RemoveFileMutex* when the file is deleted.

The most significant changes for part 2 are the async calls. The *CallbackList* gRPC function is employed to retrieve any updated list when it is available. The *Request* input type must contain a name variable to be assigned as empty later in the process. Similarly, the client will compare the *crc* and *mtime* of each file in the list to determine if any gRPC call is necessary to update the local cached files.

Since both *HandleCallbackList* and *InotifyEventCallback* will touch the files in the directory and make changes if necessary, there is a potential race condition issue for the directory modification. So a directory mutex is introduced to allow only one function to perform file writing/reading at the same time.

3.4 Testing

Testing includes:

- Test each gRPC method with an existing filename and a reasonable deadline, expecting an *OK* code.
- Test Store method with an local older version of file, expecting a *ALREADY_EXISTS* code.
- Test Fetch method with a local newer version of file, expecting a *ALREADY_EXISTS* code.
- Test Store and Delete method to validate lock acquisition and release during write operations.
- Test Mount command to update the server with local modified files.
- Test Mount command to retrieve the newest version of files from the server.

4 REFERENCE

P4L1 Remote Procedure Calls:

https://gatech.instructure.com/courses/270294/pages/p4l1-remote-procedure-calls?module_item_id=2666700

Official gRPC Documentation: <https://grpc.io/docs/>

gRPC C++ Examples: <https://github.com/grpc/grpc/tree/master/examples/cpp>

gRPC C++ Tutorial: <https://grpc.io/docs/tutorials/basic/cpp/>

gRPC Status Codes: <https://github.com/grpc/grpc/blob/master/doc/statuscodes.md>

gRPC Deadline: <https://grpc.io/blog/deadlines/>

Protocol Buffers Documentation: <https://protobuf.dev/>

Protobuffers Scalar types: <https://developers.google.com/protocol-buffers/docs/proto3#scalar>

Filesystem library: <https://en.cppreference.com/w/cpp/filesystem>

Indrasiri, Kasun, and Danesh Kuruppu. GRPC: Up and Running: Building Cloud Native Applications with Go and Java for Docker and Kubernetes. O'Reilly Media, 2020. Accessed 21 November 2024.

Tanenbaum, Andrew S., and Maarten van Steen. Distributed systems. Prentice Hall, 2002. Accessed 21 November 2024.